

PLATAFORMAS BIG DATA

3º de Ingeniería Informática

Práctica 5

Sistemas de memoria compartida

Contexto

En la pasada práctica, el alumno evaluó diferentes implementaciones de una gemm y la relación de rendimiento frente al pico de la máquina.

La presente práctica se centrará en la programación en sistemas de memoria compartida; esto significa que los hilos que lancemos a la máquina van a ver y compartir la misma memoria principal del sistema.

Para llevar a cabo el desarrollo de esta práctica se utilizará la biblioteca OpenMP y habrá que tener siempre en cuenta incluir en la cabecera del fichero `"#include <omp.h>"` así como la opción `-fopenmp` en la compilación del binario.

Objetivos

Los objetivos principales de esta práctica serán:

- El alumno debe aprender a aplicar las principales técnicas de sincronización de hilos.
- Controlar las sentencias principales que ofrece OpenMP para la paralelización de secciones de código.
- Aplicar técnicas de reducción de resultados fundamentales en plataformas de Big Data y procesamiento paralelo.

EJERCICIO 1

Este primer ejercicio consistirá en llevar a cabo una implementación paralela del producto matricial desarrollado por el alumnado en la red neuronal MLP (la versión original, sin bloques).

Una vez desarrollado, se llevará a cabo un estudio de escalabilidad. Para esto, se elegirá un tamaño de batch suficientemente grande para que todos los hilos tengan trabajo suficiente. A continuación, se observará el comportamiento del sistema con lanzamientos paralelos; para ello, se fijará el número de hilos al mismo número de núcleos del sistema y se llevarán a cabo lanzamientos contrastando el resultado con htop/top... Una vez analizado el comportamiento de la aplicación y el sistema, se deberá llevar a cabo el estudio de escalabilidad temporizador y aumentando progresivamente el número de hilos involucrados en la ejecución.

Para observar la aceleración, se obtendrá la métrica de aceleración llamada speed-up.

Speed-up = $T(s)$ versión base / $T(s)$ versión acelerada

Esto proporcionará el factor de aceleración respecto a la versión, en este caso secuencial. El speed-up ideal, que se busca en la paralelización de un algoritmo, es lineal respecto al número de núcleos involucrados en la ejecución. Esto implica que para 1 hilo la aceleración será 1, para 2 hilos 2, para 3 hilos 3, etc.

Finalmente, lleve a cabo el estudio de los resultados.

EJERCICIO 2

La paralelización del producto matricial, sin empaquetado, como ha podido observar el alumnado, es un proceso trivial con OpenMP, mediante una única directiva somos capaces de paralelizar el trabajo que se lleva a cabo dentro de varios bucles. Pero, ¿qué bucle has decidido paralelizar? ¿Porque? ¿Has probado otro?

Lleva a cabo este estudio observando la diferencia de rendimiento entre las implementaciones realizadas.

EJERCICIO 3

Una vez contestadas las preguntas, se plantea al alumno la paralelización del siguiente código secuencial buscando la máxima escalabilidad / rendimiento.

El alumno deberá llevar a cabo su propio programa, junto con las reservas de memoria pertinentes y la declaración de variables para que sea totalmente funcional.

Nota: Como una complicación extra, la paralelización deberá llevarse a cabo de manera manual, es decir, se crearán previamente los hilos con una sección

```
#pragma omp parallel
```

Y posteriormente mediante el identificador de hilo se llevará a cabo el reparto de la faena. Esto implica que no se podrá hacer uso de las cláusulas como: `parallel for`, `reduction`, etc.

```
for (long i = 0; i < N; i++) { suma += array[i]; }
```